

Beam / rigid-body coupling in beamFoam and moorFV

Revised 30 September 2026 (supersedes the earlier version of this file)

Author Colm Mc Alister (developed with Codex)

beamFoam `colm_6D0F_solver` → `openfoam-v2306`, PR #49 (draft) **moorFV** `feature-beamFoamCoupling-colm` → `main`, PR #16 (draft)

Goal. When a rigid body is attached to the end of a beam, solve the body's motion inside beamFoam's own linear system (monolithically), instead of taking turns between beamFoam and the `sixDoFRigidBodyMotion` solver.

Status (from reading the code). The goal is **not yet achieved**. In the stable configuration (`staged` mode), which both test cases use, the body is still integrated by moorFV's own Newmark solver, with the beam force taken from the previous beam iteration. Nothing couples beam and body inside the matrix. The "monolithic" and loop cases therefore do the same work: identical iteration counts, body positions equal to about $1e-10$ m. The implicit coupling terms exist, but only in the experimental `fullyImplicit` mode, and even there moorFV overwrites the body's position afterwards.

Correction. The first version of this PDF (commit `bda04a6`) described the solver internals from the Codex development notes. It said beamFoam solves the body in the same system and that this solution drives the body. A line-by-line check of the code showed that isn't true for the configuration being run. This version is based on the code at `fb5d6c1` (beamFoam) and `f024a62` (moorFV), with file and line references so each point can be checked.

1. What actually happens in one time step (staged mode)

- 1. moorFV starts its Newmark step.** `FvBeamNewmark::solve()` calls `updateAcceleration()`, which sets $a = F_{fluid}/m$ and then calls `applyRestrains()` → `finiteVolumeBeam::restrain()`.
`FvBeamNewmark.C:83; sixDoFRigidBodyMotionFvBeam.C:319–321`
- 2. The restraint sends the body state to beamFoam and runs the beam.** It passes current and previous displacement, orientation, v , a , π , τ , mass and attachment point in `RigidBodyStepData`, then calls `beam.evolve()`.
`finiteVolumeBeam.C:304–349`
- 3. The beam end is prescribed from the body**, exactly as in the loop method: before each Newton iteration, the attachment W is set to the body's current displacement ("staged target").
`Evolve.C:241–254 and 930–955`
- 4. Each Newton iteration builds the extended matrix.** The beam force going to the body is $-Q$ at the attachment face, from the *previous* Newton iterate.
`Evolve.C:378–404`
- 5. The body block in the matrix is a plain identity.** The solver is constructed with `rigidBodyKinematicCoupling = blockEigenKinematicCouplingFullyImplicit`, which is `false` in staged mode, so every beam↔body coupling block is skipped. The monolithic run's log shows "BlockEigen kinematic coupling terms added" **0 times**.
`Evolve.C:859; BlockEigenSolverOF.C:332–436`

6. **So the body "solve" is an explicit Newmark update.** Identity rows return the right-hand side: $x_{n+1} = x_n + \Delta t \cdot v_n + \Delta t^2 [\beta \cdot a + (\frac{1}{2} - \beta) \cdot a_n]$, with $a = a_{\text{fluid}} + F_{\text{beam}}/m$. Velocity, angular momentum, acceleration and torque are computed directly from that force, outside the matrix.

BlockEigenSolverOF.C:627–711, 886–899

7. **Back in moorFV, the BlockEigen state is applied** (`applyBlockEigenSolution`) and the restraint force is set to zero.

finiteVolumeBeam.C:367, 414; sixDoFRigidBodyMotionFvBeam.C:229–253

8. **...and then mostly overwritten.** `updateAcceleration()` relaxes a and τ (`accelerationRelaxation 0.5`). Then `FvBeamNewmark::solve()` **recomputes v , π , centre of rotation and orientation** from its own Newmark formulas. Only the acceleration and torque from beamFoam survive.

sixDoFRigidBodyMotionFvBeam.C:323–332; FvBeamNewmark.C:86–114

Loop (`rigidBodyAndBeam_loop`)

- Body integrated by `FvBeamNewmark`.
- Beam end prescribed from the body.
- Beam force added to the body as a restraint force ($a \ += \ rF/m$), taken from the converged beam.
- Moment: $rP \times rF$ only.

"Monolithic" (`rigidBodyAndBeam_monolithic`)

- Body integrated by `FvBeamNewmark`.
- Beam end prescribed from the body.
- Beam force added to the body inside beamFoam ($a = a + F/m$), taken from the previous Newton iterate.
- Moment: $r \times F$ plus the beam's end moment $-M$.

These three small differences are the only ones. Both are the same partitioned scheme with one exchange per time step.

2. What the implicit coupling code does (`fullyImplicit` mode, experimental)

The matrix coupling terms are implemented and are used only when `blockEigenKinematicCouplingMode fullyImplicit` is set (BlockEigenSolverOF.C:332–436; coefficients prepared in Evolve.C:323–370):

$$\begin{bmatrix} K_{\text{beam}} & K_{\text{beam} \rightarrow \text{rb}} \\ K_{\text{rb} \rightarrow \text{beam}} & I + \alpha K \end{bmatrix} \begin{bmatrix} \Delta W, \Delta \theta \text{ (beam)} \\ \Delta x_{\text{rb}}, \Delta \theta_{\text{rb}} \text{ (body)} \end{bmatrix} = \begin{bmatrix} \text{beam RHS} \\ \text{Newmark body RHS} \end{bmatrix}$$

- **Beam → body** (beam attachment rows): translation $C_w/p\Delta t$; rotation $(C_w \cdot -\text{spin}(r))/p\Delta t$.
- **Body → beam** (body translation rows only): $\alpha K \cdot (\Delta x_{\text{rb}} + \Delta \theta_{\text{rb}} \times r - \Delta W_{\text{beamCell}})$, with $\alpha K = \Delta t^2 \cdot \beta \cdot (1/m) \cdot CQW/p\Delta t$, the Newmark-scaled Hooke's-law term.
- **No implicit torque rows:** the rotation rows stay identity, and torque is explicit.

Even when this mode works, three things stop it from being authoritative:

- `FvBeamNewmark` still overwrites the solved position and orientation (step 8 above).
- The reported velocity and acceleration come from the explicit force, not from the solved displacement.
- The Codex notes record convergence problems in this mode, and it hasn't been validated.

3. Configuration switches: what each one really does

Switch (file)	Values (default first)	Effect in the code
<code>blockEigenForceCoupling</code> <code>beamProperties</code>	<code>false</code> / <code>true</code>	Adds the beam's attachment force $-Q$ (previous Newton iterate) to the body's acceleration and torque inside <code>beamFoam</code> . Explicit.
<code>blockEigenKinematicCoupling</code> <code>beamProperties</code>	<code>false</code> / <code>true</code>	Lets <code>beamFoam</code> own the beam end. In staged mode it only prescribes the end from the body's current displacement; no matrix terms . Matrix coupling blocks only in <code>fullyImplicit</code> .
<code>blockEigenKinematicCouplingMode</code> <code>beamProperties</code>	<code>staged</code> / <code>fullyImplicit</code>	Staged: no implicit coupling. Fully implicit: coupling blocks on (experimental).
<code>beamAttachmentKinematics</code> <code>dynamicMeshDict</code>	<code>prescribedByRigidBody</code> / <code>solvedByBlockEigen</code>	Whether <code>moorFV</code> or <code>beamFoam</code> sets the beam end.
<code>blockEigenAuthoritativeMotion</code> <code>dynamicMeshDict</code>	<code>false</code> / <code>true</code>	Applies the <code>BlockEigen</code> state and zeroes the restraint force. <code>FvBeamNewmark</code> then recomputes v , π , position and orientation, so only a and τ take effect .

With all defaults the code behaves as before. `solvedByBlockEigen` without kinematic coupling stops with a fatal error.

4. What changed in each repository

beamFoam (PR #49)

File	Change
<code>BlockEigenSolverOF.C/.H</code>	Appends 6 body unknowns (identity block); explicit Newmark body RHS from the beam force; beam $\leftrightarrow$ body coupling blocks (used only in <code>fullyImplicit</code>); returns a <code>RigidBodySolution</code> ; writes <code>BlockEigenSolve_rigidBodyMotion.dat</code> / ... <code>ForceCoupling.dat</code> .
<code>RigidBodyCouplingData.H</code> (new)	<code>RigidBodyStepData</code> (body state in), <code>RigidBodySolution</code> (out), <code>RigidBodyForceCoupling</code> .
<code>beamModel.H</code>	Virtuals <code>setRigidBodyStepData()</code> / <code>getRigidBodySolution()</code> .
<code>coupledTotalLagNewtonRaphsonBeam*</code>	Reads the switches, sets the staged attachment target before assembly, prepares coupling coefficients, configuration checks.
<code>original_BlockEigenSolverOF.C/.H</code>	Backup of the pre-change solver; not compiled.
<code>tutorials/</code>	<code>rigidBodyAndBeam_monolithic</code> and <code>rigidBodyAndBeam_loop</code> comparison cases; older development cases grouped in <code>rigidBodyAndBeam/</code> ; comparison and efficiency plotting scripts.

moorFV (PR #16)

File	Change
<code>finiteVolumeBeam.C/.H</code>	Builds and passes <code>RigidBodyStepData</code> ; <code>beamAttachmentKinematics</code> option; applies the BlockEigen state and zeroes the restraint force when <code>blockEigenAuthoritativeMotion</code> is on; new history files.
<code>sixDoFRigidBodyMotionFvBeam.C/.H/I.H</code>	<code>applyBlockEigenSolution()</code> ; <code>state0()</code> accessor; <code>initialCentreOfRotation()</code> made public.
<code>sixDoFRigidBodyMotionFvBeamRestraint.H</code>	<code>restrain(...)</code> now takes a non-const motion object.
<code>Make/options</code> (x2)	beamFoam headers via <code>\$(BEAMFOAM_DIR)</code> (hard-coded path removed in 74005ba).

5. Benchmark

`rigidBodyAndBeam_monolithic` against `rigidBodyAndBeam_loop` , run side by side on 30 Sep at fb5d6c1 .
Single beam, single floating box, serial, $t = 0 \rightarrow 0.25$ s, `nOuterCorrectors` 1 .

Case	Time steps	Beam Newton its	Linear solver its	Run time	Final CoR height
"Monolithic" (staged)	88	439	2863	351 s	-0.2923609842 m
Loop	88	439	2863	351 s	-0.2923609842 m

- The identical counts are what section 1 predicts: the two cases run the same partitioned scheme.
- The regression check (`Alltest`) passes, but it compares two copies of the same explicit Newmark formula fed the same force. It shows the formulas are consistent, not that the coupling is implicit.

6. What is needed for a truly monolithic solve

1. **Switch on the coupling blocks:** make `fullyImplicit` converge. That means consistent increment vs total variables, checked signs of the body→beam block, and implicit torque rows using the moment of inertia.
2. **Stop moorFV overwriting the solution:** when BlockEigen owns the motion, `FvBeamNewmark` must not recompute v , π , position and orientation after `restrain()` , or the restraint must run after the Newmark update.
3. **Make the body state consistent:** recover acceleration and velocity from the solved displacement through the Newmark relations, instead of computing them separately from the explicit force.
4. **Test where it matters:** a case where the loop needs several outer iterations or goes unstable (lighter body, stiffer line), with `nOuterCorrectors` > 1 for the loop.

7. Before merging

	Repo	Item
done	moorFV	Hard-coded <code>/Users/...</code> include paths replaced with <code>\$(BEAMFOAM_DIR)</code> (74005ba).
done	beamFoam	Codex notes moved into <code>codexLogs/</code> ; tutorial cases reorganised (fb5d6c1).
to do	both	Decide how to present the feature: as partitioned coupling inside beamFoam (what it is now), or finish the monolithic path (section 6) first.
to do	moorFV	Bring the branch up to date with <code>main</code> (82 commits behind). <code>main</code> renamed <code>sDoFRGBFvBeam</code> → <code>sixDoFRigidBodyBeamMotion</code> , and the beamFoam submodule conflicts.
to do	moorFV	<code>displacement<name>.dat</code> writes the <i>anchor</i> displacement.
to do	beamFoam	Remove generated tutorial output, <code>original_BlockEigenSolverOF.C/.H</code> and <code>#fixedDisplacementFvPatchVectorField.H#</code> .
to do	both	Verbose <code>Info</code> behind a debug switch; remove commented-out code and the unused <code>PstreamReduceOps.H</code> include.
to do	beamFoam	PR #49 also carries the <code>openfoam-v2306-seevani</code> work; merge that first or accept it bundled.

8. Codex logs in this directory

File	Date	Contents
BLOCK_EIGEN_SOLVER_COUPLING_CHANGES.md	Apr–May	Reciprocal body-row coupling terms; Newmark RHS; moment arm.
BLOCK_EIGEN_RIGID_BODY_COUPLING_PROGRESS.md	7 May	Applying the BlockEigen state in moorFV ("authoritative").
BEAM_ATTACHMENT_KINEMATICS_OWNERSHIP.md	7 May	Who sets the beam end, and why double ownership caused instability.
rigidBodyKinematicCouplingDevelopment.txt	Jul–Aug	<code>beamAttachmentKinematics</code> switch, config checks.
rigidBodyAndBeamCoupledSolverDevelopmentLog.txt	28 Jul	Diagnostics, staged mode, the $t = 0.02$ divergence fix.
rigidBodyImplicitAttachmentCoefficientChanges.txt	11 Aug	Hooke's-law attachment coefficient; sign convention.

Read these notes with care. Codex wrote them during development. They do say the beam force enters the body explicitly, and that only `fullyImplicit` keeps the matrix coupling columns active. But they call the result "authoritative", and none of them notes that `FvBeamNewmark` recomputes the body's position, velocity and orientation after the BlockEigen state is applied. Treat section 1 of this document as the reference for what the code does.